

Фильтр Цветения RocksDB

[Прыжок на дно](#)Мерлин отредактировал эту страницу 1 сентября 2021 г. · [49 правок](#)

🔗 Что такое фильтр Блума?

Для любого произвольного набора ключей можно применить алгоритм для создания битового массива, называемого фильтром Блума. По заданному произвольному ключу этот битовый массив можно использовать, чтобы определить, *может ли* или *точно не может* этот ключ существовать в наборе ключей. Более подробное объяснение принципа работы фильтров Блума можно найти в этой [статье Википедии](#).

В RocksDB, если задана политика фильтрации, каждый вновь создаваемый SST-файл будет содержать фильтр Блума, который используется для определения того, может ли файл содержать искомый ключ. Фильтр по сути представляет собой битовый массив. К заданному ключу применяется несколько хеш-функций, каждая из которых указывает на бит в массиве, который будет установлен в 1. При чтении к поисковому ключу также применяются те же хеш-функции, проверяются биты, то есть выполняется пробное вычисление, и ключ определенно не существует, если хотя бы одно из пробных вычислений возвращает 0.

🔗 Основы настройки

Пример настройки фильтра Bloom:

```
rocksdb::BlockBasedTableOptions table_options;
table_options.filter_policy.reset(rocksdb::NewBloomFilterPolicy(10,
false));
my_cf_options.table_factory.reset(
    rocksdb::NewBlockBasedTableFactory(table_options));
```



При этом создаются фильтры, использующие около 10 бит на ключ, что хорошо подходит для многих рабочих нагрузок, хотя количество бит на ключ можно плавно регулировать в диапазоне от 1 до 20 и более. Преимущество более высокого значения в том, что оно снижает нагрузку на ввод-вывод, процессор и блочный кэш, связанную с DB::Get или Seek с [префиксом Bloom](#). Преимущество более низкого значения в том, что оно уменьшает объем фильтра Блума, большая часть которого будет храниться в памяти и на диске. При настройке Bloom-фильтра важно помнить о снижении эффективности (или нелинейности; подробнее в следующих разделах):

- 1,5 бита на ключ (частота ложных срабатываний 50%) на 50% эффективнее, чем 100 бит на ключ
- 2,9 бита на ключ (частота ложных срабатываний 25%) на 75% эффективнее, чем 100 бит на ключ
- 4,9 бита на ключ (частота ложных срабатываний 10%) на 90% эффективнее, чем 100 бит на ключ
- 9,9 бита на ключ (частота ложных срабатываний 1%) на 99% эффективнее, чем 100 бит на ключ
- 15,5 бит на ключ (частота ложноположительных срабатываний 0,1%) на 99,9% эффективнее, чем 100 бит на ключ.

Использование фильтров Блума в целом требует небольших затрат ресурсов процессора, которые в значительной степени не зависят от количества бит на ключ, но для большинства рабочих нагрузок экономия намного превышает затраты. Если стоит вопрос о том, включать фильтры Блума или нет, и при этом есть опасения по поводу нехватки памяти, то лучше сравнить NewBloomFilterPolicy(3) с отсутствием политики фильтрации, чем NewBloomFilterPolicy(10) с отсутствием политики фильтрации. (Не стоит сразу предполагать, что стоит в три раза увеличить объем памяти для фильтров Блума, чтобы получить на треть больше пользы.)

🔗 Ленточный фильтр

Новая альтернатива фильтру Блума доступна в качестве замены (начиная с версии 6.15.0). Она экономит около 30 % места, занимаемого фильтром Блума (что наиболее важно для памяти), но при этом требует в 3–4 раза больше ресурсов процессора для работы с фильтрами. Большая часть дополнительного времени процессора тратится на фоновые задания по созданию фильтров, и это, как правило, оправданно, поскольку фильтры SST обычно используют около 10 % системной оперативной памяти и менее 1 % ресурсов процессора.

В настоящее время новый API предполагает, что вам нужен такой же процент ложных срабатываний, как у фильтра Блума, но при этом вы хотите сэкономить память. Например, rocksdb::NewRibbonFilterPolicy(9.9) имеет тот же процент ложных срабатываний 1%, что и фильтр Блума, но использует всего около 7 бит на ключ.

При создании каждого ленточного фильтра используется больше временной (не отслеживаемой) памяти, чем при использовании фильтров Блума (~231 бит на ключ против ~74 бит на ключ), но для компенсации этой дополнительной памяти за счет экономии на созданных фильтрах требуется всего ~50 фильтров (файлов SST или разделов). В некоторых случаях создание ленточных фильтров может замедлить пиковую скорость записи. (Мы работаем над решением этих проблем.)

Ленточные фильтры описаны в [этой статье](#).

🔗 Жизненный цикл

При настройке в RocksDB каждый SST-файл создается с помощью фильтра Блума, встроенного в сам SST-файл. Фильтры Блума обычно создаются для файлов на всех уровнях одинаково, за исключением того, что последний уровень можно пропустить, задав `optimize_filters_for_hits`, а переопределение

`FilterPolicy::GetBuilderWithContext` [обеспечивает возможность настройки](#).

Из-за различий в размерах и других ограничений фильтров Блума SST-фильтры Блума, как правило, несовместимы друг с другом для оптимизированной композиции по принципу «И» или «ИЛИ». Даже при объединении двух SST-файлов новый фильтр Блума создается с нуля с использованием ключей из нового файла — отчасти для того, чтобы обеспечить предсказуемое количество битов на ключ и, следовательно, снизить количество ложных срабатываний.

Когда мы открываем SST-файл, соответствующий фильтр Блума или Риббона также открывается и загружается в память с использованием исходных параметров конфигурации. (Любой `NewBloomFilterPolicy` или `NewRibbonFilterPolicy` будет считывать оба типа фильтров.) При закрытии SST-файла фильтр удаляется из памяти. Чтобы сохранить фильтр Блума в блочном кэше, используйте:

```
BlockBasedTableOptions::cache_index_and_filter_blocks=true, .
```

🔗 Блочный фильтр Блума (старый формат)

С помощью этого API фильтр Блума можно было создать только в том случае, если все ключи помещались в память. С другой стороны, распределение ключей между фильтрами Блума не влияет на общий процент ложных срабатываний, если размер каждого фильтра соответствует количеству добавленных ключей. Поэтому, чтобы снизить нагрузку на память при создании файла SST, в старом формате для каждого блока значений ключей размером 2 КБ создавался отдельный фильтр Блума. Подробнее о формате можно узнать [здесь](#). В конце в файл SST сохраняется массив смещений отдельных блоков фильтра Блума.

Во время чтения смещение блока, который может содержать ключ/значение, определяется по индексу SST. Затем на основе смещения загружается соответствующий фильтр Блума. Если фильтр показывает, что ключ может существовать, выполняется поиск ключа в фактическом блоке данных.

🔗 Full Filters (new format)

The individual filter blocks in the old format are not cache aligned and could result into a lot of cache misses during lookup. Moreover although negative responses (i.e., key does not exist) of the filter saves the search from the data block, the index is already loaded and looked into. The new format, full filter, addresses these issues by creating one filter for the entire SST file. The tradeoff is that more memory is required to cache a hash of every key in the file (versus only keys of a 2KB block in the old format).

Full filter limits the probe bits for a key to be all within the same CPU cache line. This ensures fast lookups and maximizes CPU cache effectiveness by limiting the CPU cache misses to one per key (per filter). Note that this is essentially sharding the bloom space and has only a small effect on the false positive rate as long as there are many keys. Refer to "The Math" section below for more details.

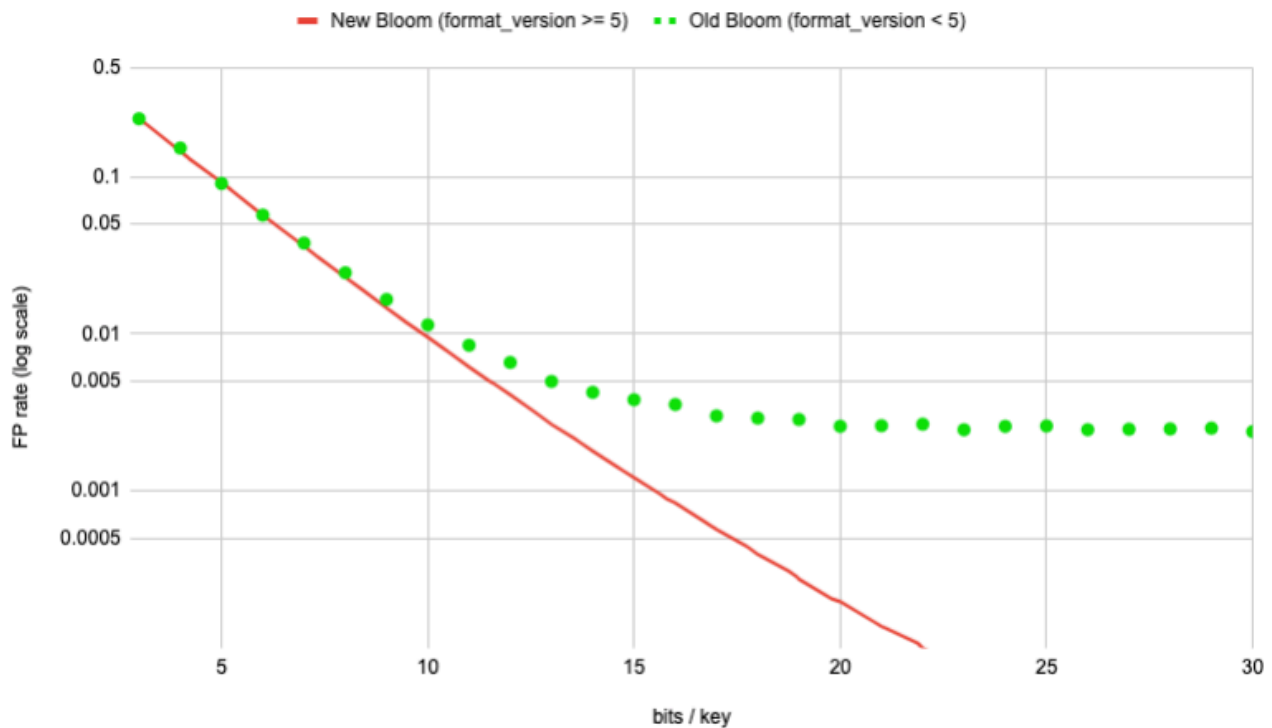
At read time, RocksDB uses the same format that was used when creating the SST file. Users can specify the format for the newly created SST files by setting `filter_policy` in options. The helper function `NewBloomFilterPolicy` can be used to create both old block-based and new full filter (the default).

```
extern const FilterPolicy* NewBloomFilterPolicy(  
    double bits_per_key, bool use_block_based_builder = false);  
}
```



The first underlying Bloom filter implementation used for full filters (and partitioned filters) had some flaws, and a new implementation is used with `format_version = 5` (version 6.6) and later. First, the original full filter could not get an FP rate better than about 0.1%, even at 100 bits/key. The new implementation is below 0.1% FP rate with only 16 bits/key. Second, possibly unlikely, the original full filter would have degraded FP rates with millions of keys in a single filter, because of inherent limitations of 32-bit hashing. The new implementation easily scales to many billions of keys in a single filter, thanks to a 64-bit hash. Aside from extra intermediate data during construction (64 bits per key rather than 32), the new implementation is generally faster also.

The following graph can be used to pick a `bits_per_key` setting to achieve a certain false positive rate in the Bloom filter, old or new implementation. You might also consult this for migrating to `format_version=5` if you wish to save filter space and keep the same FP rate. (Note that FP rates are not percentages here. For example, 0.001 means 1 in 1000 queries of keys not in the SST file will falsely return true, leading to extra work by RocksDB to determine the key is not actually in the SST.)



Prefix vs. whole key

By default a hash of every whole key is added to the bloom filter. This can be disabled by setting `BlockBasedTableOptions::whole_key_filtering` to false. When `Options.prefix_extractor` is set, a hash of the prefix is also added to the bloom. Since there are less unique prefixes than unique whole keys, storing only the prefixes in bloom will result into smaller blooms with the down side of having larger false positive rate. Moreover the prefix blooms can be optionally (using `check_filter` when creating the iterator) also used during `::Seek` and `::SeekForPrev` whereas the whole key blooms are only used for point lookups.

Statistic

Here are the statistics that can be used to gain insight of how well your full bloom filter settings are performing in production:

The following stats are updated after each `::Seek` and `::SeekForPrev` if prefix is enabled and `check_filter` is set.

- `rocksdb.bloom.filter.prefix.checked : seek_negatives + seek_positives`
- `rocksdb.bloom.filter.prefix.useful : seek_negatives`

The following stats are updated after each point lookup. If `whole_key_filtering` is set, this is the result of checking the bloom of the whole key, otherwise this is the result of checking the bloom of the prefix.

- `rocksdb.bloom.filter.useful : [true] negatives`
- `rocksdb.bloom.filter.full.positive : positives`
- `rocksdb.bloom.filter.full.true.positive : true positives`

So the observed false positive rate of point lookups is computed by $(\text{positives} - \text{true positives}) / (\text{positives} - \text{true positives} + \text{true negatives})$, which is (number of queries **returning positive** on keys or prefixes not in the SST) / (number of queries on keys or prefixes not in the SST).

Pay attention to these confusing scenarios:

1. If both `whole_key_filtering` and `prefix` are set, prefix are not checked during point lookups.
2. If only the `prefix` is set, the total number of times prefix bloom is checked is the sum of the stats of point lookup and seeks. Due to absence of true positive stats in seeks, we then cannot have the total false positive rate: only that of of point lookups.

🔗 Customize your own FilterPolicy

`FilterPolicy` (`include/rocksdb/filter_policy.h`) can be extended to define custom filters. The two main functions to implement are:

```
FilterBitsBuilder* GetFilterBitsBuilder()
FilterBitsReader* GetFilterBitsReader(const Slice& contents)
```



In this way, the new filter policy would function as a factory for `FilterBitsBuilder` and `FilterBitsReader`. `FilterBitsBuilder` provides interface for key storage and filter generation and `FilterBitsReader` provides interface to check if a key may exist in filter.

A newer alternative to `GetFilterBitsBuilder()` is also available (override only one of the two):

```
FilterBitsBuilder* GetBuilderWithContext(const FilterBuildingContext&)
```



This variant allows custom filter configuration based on contextual information such as table level, compaction style, and column family. See [LevelAndStyleCustomFilterPolicy](#) for an example selecting between configurations of built-in filters.

Notice: This builder/reader interface only works for full and partitioned filters (new format).

🔗 Partitioned Bloom Filter

Partitioned filters use the same filter block format as full filters, but use many filter blocks per SST file partitioned by key range. This feature increases the average CPU cost of an SST Bloom query because of key range partitioning, but (with `cache_index_and_filter_blocks=true`) greatly reduces worst-case data loaded into the block cache for a single small-data op. This can mitigate and smooth a block cache thrashing "cliff" and improve tail latencies. Read [here](#).

🔗 Reducing internal fragmentation

A key cost of Bloom and Ribbon filters is the memory space they occupy, and that space can often be reduced by setting `BlockBasedTableOptions::optimize_filters_for_memory` to true. This works by adjusting the bits per key on a filter-by-filter basis to minimize memory internal fragmentation. For example, with setting 8 bits per key and 900 keys going into a filter, the memory allocator is likely to return 1024 bytes of space. (Jemalloc averages about 10% internal fragmentation based on its built-in allocation sizes.) Without `optimize_filters_for_memory`, we would only generate a ~900 byte filter. With `optimize_filters_for_memory`, we would use (most of) the extra 124 bytes to reduce the FP rate of the filter, requiring no additional space in memory. For convenience, the code targets the same overall FP rate with or without `optimize_filters_for_memory`, by remembering the balance of past decisions and gently favoring toward the overall target FP rate, rather than always "rounding up". See [table.h](#) for more detail.

🔗 The math

Refer e.g. to [the wikipedia article](#) for standard false positive rate of a Bloom filter.

The CPU cache-friendly Bloom filter variant is presented and analyzed in [this 2007 paper](#) (page 4, formula 3). In short, the Bloom filter is sharded into s shards, each of size m/s . If s were much smaller than n (number of keys), each shard would have approximately n/s keys mapping to it, and the false positive rate is like a standard Bloom filter using m/s for m and n/s for n . This yields essentially the same false positive rate as for a standard Bloom filter with the original m and n .

For CPU-cache locality of probes, s is within a couple orders of magnitude of n , which leads to noticeable variance in the number of keys mapped to each shard. (This is the phenomenon behind "clustering" in hashing.) This matters because the false positive rate is non-linear in the number of keys added. Even though the median false positive rate of a shard is the same as a standard Bloom filter, what matters is the mean false positive rate, or the expected value of the false positive rate of a shard.

This can be understood using the [Poisson distribution](#) ([as an approximation of a binomial distribution](#)), using parameter n/s , the expected number of keys mapping to each shard. You can compute the expected/effective false positive rate using a summation based on the pdf of a Poisson distribution (in 2007 paper referenced above), or you can use an approximation I have found to be very good:

- The expected false positive rate is close to the average of the false positive rate for one standard deviation above and below the expected number of keys per shard.

The standard deviation is the square root of that n/s parameter. In a standard configuration, $m/s = 512$ and $m/n = 10$, so $n/s = 51.2$ and $\text{sqrt}(n/s) = 7.16$. With $k=6$, we use the standard Bloom filter false positive rate formula, substituting 51.2 ± 7.16 for n and 512 for m , yielding 0.43% and 1.48%, which averages to 0.95% for the CPU cache local Bloom filter. The same Bloom filter without cache locality has false positive rate 0.84%.

This modest increase in false positive rate, here a factor of 1.13 higher, is considered quite acceptable for the speed boost, especially in environments heavily utilizing many cores. (Ribbon filters generally offer a better space-for-time trade-off than standard Bloom filters with no locality.)

► Pages 167

Contents

- [RocksDB Wiki](#)
- [Overview](#)
- [RocksDB FAQ](#)
- [Terminology](#)
- [Requirements](#)
- [Contributors' Guide](#)
- [Release Methodology](#)
- [RocksDB Users and Use Cases](#)
- [RocksDB Public Communication and Information Channels](#)
- [Basic Operations](#)
 - [Iterator](#)
 - [Prefix seek](#)
 - [SeekForPrev](#)
 - [Tailing Iterator](#)
 - [Compaction Filter](#)
 - [Multi Column Family Iterator](#)
 - [Read-Modify-Write \(Merge\) Operator](#)
 - [Column Families](#)
 - [Creating and Ingesting SST files](#)
 - [Single Delete](#)

- [SST Partitioner](#)
- [Low Priority Write](#)
- [Time to Live \(TTL\) Support](#)
- [Transactions](#)
- [Snapshot](#)
- [DeleteRange](#)
- [Atomic flush](#)
- [Read-only and Secondary instances](#)
- [Approximate Size](#)
- [User-defined Timestamp](#)
- [Wide Columns](#)
- [BlobDB](#)
- [Online Verification](#)
- **Options**
 - [Setup Options and Basic Tuning](#)
 - [Option String and Option Map](#)
 - [RocksDB Options File](#)
- [MemTable](#)
- [Journal](#)
 - [Write Ahead Log \(WAL\)](#)
 - [Write Ahead Log File Format](#)
 - [WAL Recovery Modes](#)
 - [WAL Performance](#)
 - [WAL Compression](#)
 - [MANIFEST](#)
 - [Track WAL in MANIFEST](#)
- **Cache**
 - [Block Cache](#)
 - [SecondaryCache \(Experimental\)](#)
- [Write Buffer Manager](#)
- [Compaction](#)
 - [Leveled Compaction](#)
 - [Universal compaction style](#)
 - [FIFO compaction style](#)
 - [Manual Compaction](#)
 - [Subcompaction](#)
 - [Choose Level Compaction Files](#)
 - [Managing Disk Space Utilization](#)
 - [Trivial Move Compaction](#)
 - [Remote Compaction](#)
- **SST File Formats**
 - [Block-based Table Format](#)
 - [PlainTable Format](#)
 - [CuckooTable Format](#)
 - [External Table](#)
 - [Index Block Format](#)
 - [Bloom Filter](#)
 - [Data Block Hash Index](#)
- [IO](#)
 - [Rate Limiter](#)
 - [SST File Manager](#)

- [Direct I/O](#)
- [Compression](#)
 - [Dictionary Compression](#)
- [Full File Checksum and Checksum Handoff](#)
- [Background Error Handling](#)
- [Huge Page TLB Support](#)
- [Tiered Storage \(Experimental\)](#)
- **Logging and Monitoring**
 - [Logger](#)
 - [Statistics](#)
 - [Compaction Stats and DB Status](#)
 - [Perf Context and IO Stats Context](#)
 - [EventListener](#)
- [Known Issues](#)
- [Troubleshooting Guide](#)
- [Tests](#)
 - [Stress Test](#)
 - [Fuzzing](#)
 - [Benchmarking](#)
- **Tools / Utilities**
 - [Administration and Data Access Tool](#)
 - [How to Backup RocksDB?](#)
 - [Replication Helpers](#)
 - [Checkpoints](#)
 - [How to persist in-memory RocksDB database](#)
 - [Third-party language bindings](#)
 - [RocksDB Trace, Replay, Analyzer, and Workload Generation](#)
 - [Block cache analysis and simulation tools](#)
 - [IO Tracer and Parser](#)
- **Implementation Details**
 - [Delete Stale Files](#)
 - [Partitioned Index/Filters](#)
 - [WritePrepared-Transactions](#)
 - [WriteUnprepared-Transactions](#)
 - [How we keep track of live SST files](#)
 - [How we index SST](#)
 - [Merge Operator Implementation](#)
 - [RocksDB Repairer](#)
 - [Write Batch With Index](#)
 - [Two Phase Commit](#)
 - [Iterator's Implementation](#)
 - [Simulation Cache](#)
 - [\[To Be Deprecated\] Persistent Read Cache](#)
 - [DeleteRange Implementation](#)
 - [unordered_write](#)
- **Extending RocksDB**
 - [RocksDB Configurable Objects](#)
 - [The Customizable Class](#)
 - [Object Registry](#)
- **RocksJava**
 - [RocksJava Basics](#)

- [Logging in RocksJava](#)
- [JNI Debugging](#)
- [RocksJava API TODO](#)
- [RocksJava Performance on Flash Storage](#)
- [Tuning RocksDB from Java](#)
- **Performance**
 - [Performance Benchmarks](#)
 - [In Memory Workload Performance](#)
 - [Read-Modify-Write \(Merge\) Performance](#)
 - [Delete A Range Of Keys](#)
 - [Write Stalls](#)
 - [Pipelined Write](#)
 - [MultiGet Performance](#)
 - [Tuning Guide](#)
 - [Memory usage in RocksDB](#)
 - [Speed-Up DB Open](#)
 - [Implement Queue Service Using RocksDB](#)
 - [Asynchronous IO](#)
 - [Off-peak in RocksDB](#)
- [Projects Being Developed](#)
- **Misc**
 - [Building on Windows](#)
 - [Developing with an IDE](#)
 - [Open Projects](#)
 - [Talks](#)
 - [Publication](#)
 - [Features Not in LevelDB](#)
 - [How to ask a performance-related question?](#)
 - [Articles about Rocks](#)

Clone this wiki locally

`https://github.com/facebook/rocksdb.wiki.git`

